PRM393 – Mobile Programming

Lab 03: Firebase-Powered Journal Trend Analyzer

1. Introduction

In Lab 02, students developed a Flutter application for analyzing research publication trends using the OpenAlex API.

In this lab, students will enhance the application by integrating Firebase services, automated testing, and software quality assurance practices. The objective is to gain practical experience in developing cloud-enabled mobile applications while applying modern software engineering techniques.

The application shall continue using the OpenAlex API as the primary source of publication data and integrate Firebase services to support authentication, cloud storage, push notifications, analytics, crash monitoring, and remote configuration.

---

2. Learning Objectives

Upon successful completion of this assignment, students will be able to:

- Integrate Firebase services into a Flutter application.
- Implement user authentication using Google Sign-In.
- Upload and manage files using Firebase Storage.
- Receive push notifications using Firebase Cloud Messaging (FCM).
- Track user activities using Firebase Analytics.
- Monitor application crashes using Firebase Crashlytics.
- Configure application behavior dynamically using Firebase Remote Config.
- Implement automated end-to-end testing using Patrol.
- Apply AI-assisted code review techniques to improve software quality.
- Design and develop maintainable Flutter applications using the MVVM architectural pattern.

---

3. Assignment Requirements

Students are required to enhance the Journal Trend Analyzer application developed in Lab 02.

The application must continue using OpenAlex as the primary source of publication data and Firebase as the cloud platform.

The application shall use a Bottom Navigation Bar containing four main sections:

- Home
- Journals

- Keywords
- Profile

---

4. Functional Requirements

4.1 User Authentication

The application shall authenticate users using Firebase Authentication with Google Sign-In.

Users shall be able to:

- Sign in using a Google account.
- View authenticated user information.
- Sign out from the application.

4.2 Home

The Home screen shall provide an overview dashboard for a selected research topic.

Users shall be able to:

- Search publications by topic.
- View publication trends over time.
- View total publications.
- View average citation count.
- View the most active publication year.
- View the top contributing author.
- View the top journal.
- View the most influential publication.

The publication trend shall be visualized using an appropriate chart.

Selecting a publication shall navigate to the Publication Detail screen.

4.3 Publication Details

The application shall display detailed publication information, including:

- Title
- Authors
- Publication year
- Journal name
- Citation count
- DOI
- Abstract (when available)

The application shall provide a link to the original publication when available.

4.4 Journals

The Journals screen shall provide journal-level analysis for the selected research topic.

The application shall display:

- Top journals ranked by publication count.
- Publication statistics per journal.
- Journal contribution charts.
- Citation statistics by journal.

Selecting a journal shall navigate to the Journal Detail screen.

4.5 Journal Details

The application shall display detailed information about a selected journal, including:

- Journal name
- Total number of publications.
- Total citations.
- Average citations per publication.
- Related publications.

4.6 Keywords

The Keywords screen shall provide keyword-based research analysis.

The application shall display:

- Most frequent keywords.
- Trending keywords.
- Keyword frequency statistics.
- Keyword trend charts.

Selecting a keyword shall navigate to the Keyword Detail screen.

4.7 Keyword Details

The application shall display analytical information related to a selected keyword.

The analysis shall include:

- Publication trends over time.

- Related journals.
- Related publications.
- Top contributing authors.
- Author publication counts.
- Author ranking list or chart.

Authors shall be ranked in descending order based on the number of publications associated with the selected keyword.

4.8 Profile

The Profile screen shall provide user account management and Firebase service demonstrations.

User Information

Display:

- Profile picture
- User name
- Email address

Provide:

- Sign Out

Notification Center

Display notifications received from Firebase Cloud Messaging (FCM).

Examples:

- New trending research topic.
- Highly cited publication alert.
- Research trend updates.

Report Export

Allow users to:

- Export dashboard analytics as a PDF report.
- Upload the generated report to Firebase Storage.
- Display the uploaded file URL.

Remote Config Demo

Retrieve and display at least two configuration values from Firebase Remote Config.

Examples:

- Maximum number of journals displayed.

- Maximum number of keywords displayed.

Crashlytics Demo

Provide functionality to:

- Generate a handled exception.
- Generate a test crash.

---

5. Firebase Requirements

The application must integrate the following Firebase services:

Firebase Service	Purpose
Firebase Authentication	Google Sign-In
Firebase Storage	Store exported PDF reports
Firebase Cloud Messaging (FCM)	Push notifications
Firebase Analytics	User activity tracking
Firebase Crashlytics	Crash monitoring
Firebase Remote Config	Dynamic configuration

Firebase Analytics Events

Students are required to implement and track the following Firebase Analytics events:

Event Name	Description
login	User successfully signs in
search_topic	User searches for a research topic
view_publication	User opens a publication detail page
view_journal	User opens a journal detail page
view_keyword	User opens a keyword detail page
export_pdf	User exports and uploads a PDF report
logout	User signs out

Each event should include appropriate parameters whenever applicable.

Examples:

search_topic

Parameters:

- keyword

view_publication

Parameters:

- publication_title
- publication_year

view_journal

Parameters:

- journal_name

view_keyword

Parameters:

- keyword

export_pdf

Parameters:

- topic

Evidence of recorded events must be included in the project report.

---

6. Technical Requirements

The application must:

- Be developed using Flutter and Dart.
- Consume data directly from the OpenAlex API.
- Implement asynchronous API communication.
- Handle loading states and errors properly.
- Follow a clean and maintainable project structure.
- Use appropriate state management techniques.
- Run successfully on Android devices and Android emulators.

Architecture Requirements

The application must follow the MVVM (Model–View–ViewModel) architectural pattern.

Students may use either:

- Provider
- Riverpod

for state management.

The architecture should clearly separate:

- Models
- Services
- ViewModels
- Views

Business logic should not be implemented directly inside UI screens.

Suggested project structure:

```
lib/
│
├── models/
├── services/
├── firebase/
├── viewmodels/
├── screens/
├── widgets/
└── utils/
```

---

7. User Interface Requirements

The application must contain at least the following screens:

1. Login Screen
2. Home Screen
3. Publication Detail Screen
4. Journals Screen
5. Journal Detail Screen
6. Keywords Screen
7. Keyword Detail Screen
8. Profile Screen

The application must use a Bottom Navigation Bar containing:

- Home
- Journals
- Keywords
- Profile

The user interface should be responsive, visually consistent, and easy to navigate.

---

8. Automated Testing with Patrol

Students are required to implement automated end-to-end (E2E) tests using the Patrol testing framework.

The objective is to verify critical application workflows and improve software quality through automated testing.

Required Test Scenarios

Test Case 1 – Google Sign-In

- Launch the application.
- Perform Google Sign-In.
- Verify successful navigation to the Home screen.

Test Case 2 – Topic Search

- Enter a research topic.
- Execute search.
- Verify publication results are displayed.

Test Case 3 – Publication Details

- Open a publication from the search results.
- Verify publication information is displayed correctly.

Test Case 4 – Journals Navigation

- Navigate to the Journals tab.
- Verify journal statistics and journal list are displayed.

Test Case 5 – Journal Details

- Open a journal from the journal list.
- Verify journal details are displayed correctly.

Test Case 6 – Keywords Navigation

- Navigate to the Keywords tab.
- Verify keyword statistics and keyword list are displayed.

Test Case 7 – Keyword Details

- Open a keyword from the keyword list.
- Verify keyword analysis information is displayed.

Test Case 8 – Profile Navigation

- Navigate to the Profile tab.
- Verify user profile information is displayed.

Test Case 9 – PDF Export

- Generate a PDF report.
- Upload the report to Firebase Storage.
- Verify successful upload.

Test Case 10 – Remote Config

- Retrieve Remote Config values.
- Verify configuration values are displayed.

Test Case 11 – Logout

- Perform logout.
- Verify redirection to the Login screen.

Suggested structure:

```
patrol_tests/
│
├── authentication_test.dart
├── publication_test.dart
├── journal_test.dart
├── keyword_test.dart
├── profile_test.dart
├── export_test.dart
└── remote_config_test.dart
```

Evidence required:

- Patrol test source code screenshots.
- Test execution screenshots.
- Test results summary.
- Brief explanation of each implemented test case.

---

9. AI-Assisted Code Review

Students are required to conduct an AI-assisted code review before submission.

Students may use tools such as:

- GitHub Copilot Code Review
- CodeRabbit
- SonarQube

- Kodus AI

The review must identify at least three issues, warnings, bugs, code smells, or improvement opportunities.

Students should address the findings whenever appropriate and document the review process in the project report.

Evidence of the review process must be included through screenshots and brief explanations.

---

10. Deliverables

10.1 Source Code

Students shall submit the complete source code through a GitHub repository named according to the following convention:

```
PRM393_Lab03_StudentID
```

The repository must contain:

- Complete source code
- Firebase configuration files
- Patrol test scripts
- Required assets and resources

10.2 Project Report

Students shall submit a project report of approximately 5–10 pages.

The report should include:

- Project overview
- System architecture
- MVVM implementation
- Firebase integration design
- Screenshots of implemented features
- Firebase Analytics events
- Crashlytics reports
- Remote Config demonstration
- Patrol test scenarios and results
- AI-assisted code review findings
- Challenges encountered
- Lessons learned

10.3 Demonstration Video

Students shall submit a demonstration video of approximately 5–10 minutes.

The video should demonstrate:

- Google Sign-In
- Topic search
- Publication details
- Journal analysis
- Keyword analysis
- Author analysis
- PDF export and upload
- Push notifications
- Remote Config
- Crashlytics testing
- Patrol automated testing
- AI-assisted code review

---

11. Evaluation Criteria

Criterion	Weight
Functional Requirements	30%
Firebase Integration & Analytics	25%
Architecture (MVVM + Provider/Riverpod)	10%
UI/UX and Application Quality	10%
Patrol Automated Testing	15%
AI-Assisted Code Review	5%
Report and Demonstration	5%

Total: 100%